

Principal Engineer Interview Guide

Virtual Memory, Paging, TLB, mmap, Page Cache, NUMA and Production Diagnostics

How to use this guide: Start with the short interview answer. Expand into the mechanism only when the interviewer probes. At Principal level, connect CPU behavior, kernel policy, multicore effects and production observability rather than stopping at definitions.

Core Mental Model

For most questions, keep this path in your head:

CPU instruction	Virtual address	TLB lookup	Page-table walk	Physical address	Cache / DRAM
-----------------	-----------------	------------	-----------------	------------------	--------------

A TLB miss means translation is not cached. A page fault means the current translation/protection state cannot satisfy the access and the CPU traps into the kernel. A TLB miss can be resolved entirely in hardware without a page fault.

Address Translation and Faults

1. Explain virtual memory from CPU instruction to DRAM access.

Interview answer: A load/store instruction generates a virtual address. The CPU splits it into a virtual page number and page offset. It checks the TLB for a cached virtual-to-physical translation. On a TLB hit, permission checks are applied and the physical address is formed. The access then goes through the CPU cache hierarchy; only a cache miss may finally require DRAM.

Mechanism: On a TLB miss, the hardware page-table walker reads the process page tables using the page-table root register. If a valid PTE is found, the TLB is populated and the instruction resumes. If the PTE says the page is absent or the access violates permissions, the CPU raises a page-fault exception and transfers control to the kernel.

Principal-level point: Virtual-memory translation and data-cache lookup are related but distinct. Translation does not imply a DRAM access: both page-table entries and application data may already be cached.

2. Why do we need virtual memory?

Interview answer: Virtual memory gives each process an isolated address space, enables protection, allows convenient contiguous virtual ranges over non-contiguous physical memory, supports demand paging, memory-mapped files, shared memory, Copy-on-Write and controlled overcommit.

Mechanism: It also decouples program addresses from physical placement. The OS can move/reclaim pages, share read-only code, map files, and enforce per-page read/write/execute/user permissions.

Principal-level point: The strongest answer is not 'to use more memory than RAM'. Isolation, protection and flexible mapping are fundamental even on machines that never swap.

3. What exactly is stored in a page table entry?

Interview answer: A PTE contains a physical page-frame number when the mapping is present plus control/status bits. Typical bits include present/valid, read-write, user/supervisor, executable-disable, accessed/reference, dirty, caching attributes and architecture-specific flags.

Mechanism: For a non-present entry, the OS can use software-defined bits to encode state such as swapped-out pages. Exact layout is architecture-dependent and higher-level page-table entries may point to the next page-table level or represent a huge-page mapping.

Principal-level point: Separate architectural hardware-defined bits from OS interpretation. A PTE is not simply 'virtual address -> physical address'.

4. What happens during a TLB miss?

Interview answer: The processor cannot find the translation in its translation cache, so it performs a page-table walk, typically in hardware on x86-64. If the page-table hierarchy contains a valid mapping, the translation is inserted into the TLB and execution continues.

Mechanism: Page-table-walk memory accesses themselves are cached by normal caches and specialized paging-structure caches. If the walk discovers a non-present mapping or protection violation, the miss turns into a page-fault exception.

Principal-level point: A TLB miss is a performance event; it is not automatically an OS event.

5. TLB miss vs page fault?

Interview answer: A TLB miss means 'translation not cached'. A page fault means 'the access cannot currently be completed using the page-table state/permissions'. A TLB miss normally causes a page-table walk; a page fault traps into the kernel.

Mechanism: A valid present PTE resolves a TLB miss without kernel intervention. A non-present PTE, write to a read-only COW page, or illegal permission can cause a page fault.

Principal-level point: This distinction is a common senior-level filter: TLB miss is normally much cheaper than a major page fault.

6. Explain a page fault end-to-end.

Interview answer: The CPU detects that the access cannot proceed, records fault information and traps to the kernel's page-fault handler. The kernel identifies the faulting virtual address and access type, finds the corresponding virtual-memory area, validates permissions, and decides whether the fault is legitimate.

Mechanism: For a legitimate demand-zero fault, it allocates/maps a page; for a file-backed mapping it obtains the page from page cache or storage; for COW it allocates a private page and copies data. It updates the page table, performs any required TLB invalidation, and returns so the faulting instruction is retried. If the address or permission is invalid, the kernel delivers a signal such as SIGSEGV.

Principal-level point: A page fault is a mechanism, not necessarily an error. Many faults are expected consequences of demand paging.

7. Minor vs major page fault?

Interview answer: A minor fault can be satisfied without reading the required page from storage; a major fault requires I/O to bring data in from storage. Both trap into the kernel.

Mechanism: Examples of minor faults include demand-zero allocation, COW where the source page is resident, or mapping a file page already in page cache. A major fault occurs when a needed file/swap page is not resident and storage I/O is required.

Principal-level point: Major faults are usually much more damaging to tail latency because storage latency enters the critical path.

8. What causes segmentation faults?

Interview answer: On Linux, SIGSEGV is normally delivered when a process accesses an unmapped virtual address or violates memory protection, such as writing to a read-only page or executing a non-executable mapping.

Mechanism: Typical software causes are null/dangling/wild pointers, buffer overruns that cross into an invalid page, use-after-free, stack overflow into a guard page, or writing to read-only memory. The CPU first raises a page-fault/protection exception; the kernel determines the access is invalid and sends SIGSEGV.

Principal-level point: Not every out-of-bounds access segfaults. If the bad address still lands in a valid mapped page, corruption may occur silently.

Allocation, Demand Paging and Copy-on-Write

9. Why can malloc(1 GB) succeed even when 1 GB of RAM isn't immediately consumed?

Interview answer: malloc usually reserves virtual address space and allocator metadata; it does not necessarily cause 1 GB of physical pages to be committed immediately. Linux demand paging allocates physical backing as pages are touched.

Mechanism: Large allocations are often backed by anonymous mmap regions. With overcommit, the kernel may permit reservations beyond currently free RAM depending on policy. Untouched pages therefore increase virtual size but may contribute little to RSS.

Principal-level point: Successful allocation is not proof that all future writes can be satisfied. Under overcommit, failure can be deferred until pages are actually faulted in.

10. What happens on the first access to newly allocated memory?

Interview answer: For an untouched anonymous page, the first access commonly causes a minor page fault because no private physical page is mapped yet.

Mechanism: For a read, Linux can map a shared read-only zero page in some cases. On the first write, the kernel allocates a physical page, zeroes it for security, installs a writable PTE and retries the instruction.

Principal-level point: This is why allocation benchmarks must distinguish reserve time from page-touch/first-touch time.

11. Explain demand paging.

Interview answer: Demand paging means the OS establishes virtual mappings but brings in or allocates physical pages only when they are actually accessed.

Mechanism: The first access faults; the kernel creates the needed anonymous page or obtains a file-backed page, updates the PTE, and resumes execution. This reduces startup cost and avoids consuming RAM for never-used portions of mappings.

Principal-level point: Demand paging trades eager work for fault-time latency and makes locality of reference important.

12. Explain fork() using Copy-on-Write.

Interview answer: fork creates a child with logically identical memory, but parent and child initially share the same physical pages instead of copying them. Writable private pages are marked so writes trigger COW faults.

Mechanism: When either process writes, the CPU faults because the mapping is temporarily non-writable. The kernel allocates a new page, copies the old contents, remaps the writer to its private copy, restores write permission and resumes.

Principal-level point: COW turns the cost of copying the entire address space into a pay-as-written cost.

13. Why is fork() relatively cheap initially?

Interview answer: Because the kernel mainly duplicates process metadata and page-table structures while the bulk of user memory remains shared through COW.

Mechanism: It still has costs: page-table copying/walking can be significant for very large address spaces, and subsequent writes can cause many COW faults and memory copies.

Principal-level point: For a huge memory process, 'fork is cheap' is relative; page-table overhead and post-fork COW storms can still be operationally expensive.

14. What happens when parent and child modify the same COW page simultaneously?

Interview answer: Each process has a separate page-table mapping and a write to the shared COW page faults. Kernel synchronization protects the underlying memory-management structures so the COW transition is race-safe.

Mechanism: Each writer ultimately receives a private physical page containing the pre-write contents and performs its own modification. They do not race on a single writable physical page merely because the original page was shared.

Principal-level point: The exact kernel locking/refcount implementation is internal, but the semantic guarantee is process isolation after COW.

Shared Memory, mmap and Page Cache

15. How does shared memory work across processes?

Interview answer: The OS maps the same physical page frames into the virtual address spaces of multiple processes. Each process may have a different virtual address, but its PTEs resolve to the same physical frames.

Mechanism: This can be created with mmap using MAP_SHARED, POSIX shared memory, System V shared memory, or shared file mappings. Since CPUs cache the same physical memory coherently, processes can communicate through it, but they still need synchronization such as mutexes, futexes, semaphores or atomics.

Principal-level point: Shared memory solves data sharing, not synchronization. Memory ordering and crash consistency may also matter.

16. Can two processes have the same virtual address?

Interview answer: Yes. Virtual addresses are interpreted in the context of a process address space. Address 0x400000 in process A can map to a completely different physical page than 0x400000 in process B.

Mechanism: The page-table root associated with the currently running address space determines the translation.

Principal-level point: A pointer value is therefore not globally meaningful across processes unless both intentionally establish compatible mappings.

17. Can two different virtual addresses point to the same physical address?

Interview answer: Yes. Aliasing is common. Shared libraries, shared memory, file mappings, COW before a write, or even two mappings within one process can resolve different virtual addresses to the same physical page.

Mechanism: Permissions can differ between mappings, subject to architecture/OS constraints.

Principal-level point: This is central to zero-copy designs and shared-memory IPC.

18. How does mmap() work?

Interview answer: mmap creates a virtual-memory area describing a mapping between a virtual range and a backing object or anonymous memory. It normally does not populate every page immediately.

Mechanism: On first access, page faults populate PTEs. For file-backed mappings, pages are generally served through the page cache. MAP_PRIVATE modifications use COW; MAP_SHARED modifications are visible through the shared cached page and can later be written back to the file.

Principal-level point: mmap is primarily a mapping API, not simply an allocation API.

19. mmap() vs malloc()?

Interview answer: malloc is a user-space memory-allocation interface that returns heap memory and manages blocks for the application. mmap is a kernel interface for creating virtual-memory mappings.

Mechanism: Allocators commonly obtain memory from the OS using brk/sbrk for portions of the heap and mmap for large or special allocations. mmap can map files, anonymous regions, shared memory and fixed/protected regions; malloc provides convenient variable-sized object allocation.

Principal-level point: Do not compare them as equivalent layers: malloc is an allocator policy built above OS virtual-memory primitives.

20. What is the page cache?

Interview answer: The page cache is the kernel's in-memory cache of file-backed data. It avoids repeated storage I/O and provides the cached pages used by normal buffered file I/O and most memory-mapped file accesses.

Mechanism: Reads can be satisfied from cached pages; writes usually modify cached pages and mark them dirty for later writeback.

Principal-level point: Linux often uses otherwise idle RAM for cache, so 'free memory is low' can be healthy.

21. How do memory-mapped files interact with the page cache?

Interview answer: A file-backed mmap normally maps file pages represented in the page cache. On a fault, the kernel finds or loads the relevant cached page and installs a PTE pointing to it.

Mechanism: MAP_SHARED writes dirty the shared cached page and can be written back to the file. MAP_PRIVATE writes trigger COW and create private anonymous pages rather than changing the underlying file.

Principal-level point: mmap and read() are not necessarily separate copies of file contents; they can converge on the same page-cache pages.

Reclaim, Swap and Memory Pressure

22. Why are page tables multi-level instead of one giant table?

Interview answer: A flat page table for a large virtual address space would consume enormous memory even for sparse address spaces. Multi-level tables allocate lower levels only for portions of the address space that are actually mapped.

Mechanism: The tradeoff is additional translation work on a TLB miss, mitigated by TLBs and paging-structure caches.

Principal-level point: Multi-level paging is a space-time tradeoff optimized for sparse virtual address spaces.

23. What is RSS vs VSZ?

Interview answer: VSZ is the total virtual address space mapped by a process. RSS is the amount of the process's resident mapped memory currently occupying physical RAM, though accounting of shared pages can make interpretation non-trivial.

Mechanism: VSZ includes untouched reservations, mapped files, shared libraries and other virtual regions. RSS grows when pages become resident and can fall when pages are reclaimed.

Principal-level point: For leak diagnosis, RSS alone is insufficient; use PSS/private dirty/anonymous mappings and allocator-level data.

24. What happens under memory pressure?

Interview answer: The kernel tries to free reclaimable memory. It can evict clean file-cache pages, write back dirty file pages, reclaim anonymous pages via swap if configured, shrink kernel caches, compact memory when needed, and throttle tasks in some paths.

Mechanism: If reclaim cannot satisfy demand, allocation latency rises; direct reclaim and I/O can put application threads on the critical path. Severe pressure can cause thrashing, and eventually the OOM killer may terminate processes.

Principal-level point: Memory pressure is often a latency problem before it becomes an OOM problem.

25. What is swapping?

Interview answer: Swapping moves reclaimable anonymous memory pages from RAM to a swap device/file so physical frames can be reused. Later access faults the page back in.

Mechanism: File-backed clean pages generally do not need swap because they can be discarded and reread from their file. Dirty anonymous state has no file backing, so swap provides backing storage.

Principal-level point: Swap can improve survivability but uncontrolled swap-in/out can destroy latency.

26. What is thrashing?

Interview answer: Thrashing occurs when the active working sets do not fit in available memory and the system spends excessive time reclaiming, evicting and faulting pages instead of doing useful work.

Mechanism: Symptoms include high major-fault/swap activity, high I/O wait or memory stalls, low useful CPU throughput and severe tail latency.

Principal-level point: The fix is not merely 'add swap'; reduce working set, increase RAM, fix memory growth, tune workload placement or prevent competing workloads from forcing constant reclaim.

27. How does the OS choose pages to reclaim?

Interview answer: Linux approximates recency and usage rather than maintaining a perfect LRU. It tracks reclaimable file-backed and anonymous pages, uses reference information, and prioritizes pages considered colder while balancing file cache and anonymous memory.

Mechanism: Modern Linux kernels may use multi-generational LRU (MGLRU) when enabled, grouping pages by recency generations to improve reclaim decisions. Dirty pages may require writeback before reclaim.

Principal-level point: At interview level, explain policy goals and observability rather than claiming a simplistic exact-LRU algorithm.

Multicore TLB Coherence and Huge Pages

28. What are dirty and accessed bits?

Interview answer: The accessed/reference bit indicates that a mapping has been used; the dirty bit indicates that a page has been written. Hardware commonly sets these bits in page-table entries, and the OS uses them for reclaim/accounting decisions.

Mechanism: For file-backed dirty pages, data must eventually be written back before the page can be safely discarded. Access information helps approximate which pages are hot or cold.

Principal-level point: These bits connect hardware access history to OS replacement policy.

29. What happens when the CPU modifies a page-table entry?

Interview answer: Normally the OS modifies page tables; hardware may update status bits such as accessed and dirty. When the kernel changes a translation or permissions, stale cached TLB entries must not remain usable.

Mechanism: The kernel uses architecture-specific invalidation instructions and, when other CPUs may cache the mapping, coordinates TLB shootdowns.

Principal-level point: Page-table memory updates and TLB coherence are separate concerns; updating a PTE alone is not enough.

30. What is a TLB shootdown?

Interview answer: A TLB shootdown is the mechanism used to invalidate stale translations on other CPUs when a mapping that they may have cached changes.

Mechanism: The initiating CPU updates page tables and sends inter-processor interrupts or equivalent requests to relevant CPUs. Those CPUs invalidate the required TLB entries and acknowledge completion when synchronization is required.

Principal-level point: This is one reason virtual-memory operations can become expensive at high core counts.

31. Why can TLB shootdowns hurt multicore scalability?

Interview answer: They require cross-core coordination. A mapping change can interrupt multiple CPUs, force them to execute invalidation work, create synchronization barriers and destroy useful cached translations.

Mechanism: Frequent mmap/munmap/mprotect, allocator behavior, page migration or other mapping churn can therefore create system-wide overhead on large machines.

Principal-level point: At scale, optimize not only lock contention but also coherence traffic and cross-core invalidation frequency.

32. Why do huge pages improve performance?

Interview answer: Huge pages map much more memory per TLB entry, increasing TLB reach and reducing page-table-walk frequency. They also reduce page-table memory overhead.

Mechanism: A 2-MB page covers 512 times the memory of a 4-KB page. Large working sets such as databases, JVM heaps or in-memory analytics can therefore see fewer TLB misses.

Principal-level point: Huge pages help when address-translation overhead is material; they do not automatically improve every workload.

33. What are the disadvantages of huge pages?

Interview answer: They reduce allocation granularity, can increase internal fragmentation, make contiguous physical allocation/compaction harder, can increase COW or migration costs, and may cause latency spikes when the system tries to create large contiguous pages.

Mechanism: Transparent Huge Pages can help some workloads but hurt latency-sensitive ones depending on fault/compaction behavior. Explicit huge pages require capacity planning.

Principal-level point: Treat huge pages as a workload-specific performance tool, not a universal tuning switch.

NUMA, Context Switching and Protection

34. Internal vs external fragmentation?

Interview answer: Internal fragmentation is wasted space inside an allocated unit, for example using only a small portion of a page or allocator size class. External fragmentation is free memory split into pieces such that a sufficiently large contiguous block is difficult to obtain.

Mechanism: Paging largely avoids external fragmentation for ordinary 4-KB physical allocations, but higher-order/huge-page allocations can still suffer from physical fragmentation.

Principal-level point: Allocator fragmentation and physical-memory fragmentation are different layers and should not be conflated.

35. What is NUMA and how does memory placement affect performance?

Interview answer: NUMA systems have multiple memory nodes. A CPU accesses memory attached to its local node with lower latency and often higher bandwidth than remote-node memory.

Mechanism: Linux can place pages according to first-touch and NUMA policies, and may migrate pages. If threads run on one socket while their working set lives on another, remote accesses consume interconnect bandwidth and increase latency.

Principal-level point: For large services, CPU affinity, memory policy, thread migration and first-touch initialization can matter as much as algorithmic complexity.

36. Why can a program become slow despite plenty of apparently available memory?

Interview answer: Because total free/available memory does not tell you whether the working set is local, resident, translation-efficient or free from reclaim stalls. The service may suffer NUMA remote access, page faults, TLB misses, direct reclaim, memory compaction, allocator contention, page-cache misses or cgroup limits.

Mechanism: Also inspect PSI memory pressure, swap activity, major faults, I/O latency, THP behavior and whether 'available' memory exists on the wrong NUMA node.

Principal-level point: Diagnose latency by the resource on the critical path, not by one global memory number.

37. What happens to page tables during a context switch?

Interview answer: When switching to a process with a different address space, the kernel changes the hardware page-table root to that process's page-table hierarchy. The page tables themselves are not copied on every context switch.

Mechanism: Kernel mappings may be shared or handled specially depending on architecture and security mitigations. Translation caches may retain tagged entries when ASIDs/PCIDs are available.

Principal-level point: Context-switch cost includes address-space effects, but modern CPUs try to avoid throwing away all translation state.

38. Does every context switch require flushing the TLB?

Interview answer: No. A thread switch within the same address space does not require discarding that address space's translations. Even across address spaces, processors with address-space tags such as PCIDs/ASIDs can retain translations tagged by address space.

Mechanism: Without such tagging, switching page-table roots may require broader invalidation to prevent using translations from the previous process.

Principal-level point: This is a hardware/OS cooperation optimization that materially reduces context-switch overhead.

Production Diagnostics and Scenario Questions

39. What are ASIDs/PCIDs and why are they useful?

Interview answer: They tag TLB entries with an address-space identifier so translations from multiple processes can coexist in the TLB safely. PCID is the x86 terminology; ASID is commonly used on other architectures.

Mechanism: On a context switch, the CPU can select a different address-space tag instead of flushing all old translations. Entries remain reusable when the process runs again, unless explicitly invalidated.

Principal-level point: They reduce destructive TLB churn and are increasingly valuable as TLBs and core counts grow.

40. How are memory protection permissions enforced?

Interview answer: Page-table entries encode permissions such as user/supervisor, read/write and execute-disable. During translation/access, the MMU checks the requested operation and current privilege level against those permissions.

Mechanism: A violation raises a protection/page-fault exception. The kernel may resolve special cases such as COW; otherwise it reports the illegal access to the process, commonly via SIGSEGV.

Principal-level point: Protection is enforced by hardware on every relevant memory access, with the OS controlling policy through page tables.

41. How would you diagnose excessive page faults in a production Linux service?

Interview answer: First separate minor from major faults and correlate them with latency. I would use `pidstat -r, /proc/stat`, `perf stat` page-fault events, `vmstat`, `sar`, and service-level latency metrics. For system pressure I would inspect `/proc/vmstat` and `PSI`.

Mechanism: Then identify the faulting mappings and cause: startup/demand-zero behavior, `mmap` file misses, COW, swap-ins, working-set growth, THP behavior or reclaim. `perf record/report`, eBPF tools where available, `smaps`, `numa_maps`, `iostat` and block latency can narrow the source.

Principal-level point: The key is to ask whether faults are expected minor faults or storage/reclaim-driven faults on the request critical path.

42. How would you diagnose increasing RSS?

Interview answer: I would decompose RSS by mapping rather than assuming a leak. Start with `/proc//smaps` or `smaps_rollup`, `pmap -x`, and metrics for anonymous vs file-backed, private vs shared, clean vs dirty memory.

Mechanism: Then inspect allocator statistics/profiling, thread stacks, mmap regions, caches, queues and object counts. Compare application logical live bytes with process RSS. Check whether RSS falls after allocator purge/trim or workload quiescence and whether memory is reclaimable.

Principal-level point: RSS growth is a symptom. Determine ownership and reclaimability before calling it a leak.

43. How would you distinguish a memory leak from page-cache growth or allocator fragmentation?

Interview answer: A true application leak shows increasing live/unreachable allocations or private anonymous memory tied to objects that are no longer needed. Page-cache growth is largely file-backed/reclaimable system memory. Allocator fragmentation means the allocator owns pages in RSS even though application live allocations are much smaller.

Mechanism: Use `smaps/smmaps_rollup` to classify mappings, allocator profiling such as `jemalloc/tcmalloc/glibc` diagnostics as appropriate, heap profilers, and `cgroup/system` cache metrics. Test whether allocator purge/trim reduces RSS and whether memory pressure reclaims file cache without reducing application live state.

Principal-level point: Compare three layers: application live bytes, allocator retained/resident bytes, and kernel file cache. That decomposition is more reliable than watching RSS alone.

44. A service has low CPU but extremely high latency under memory pressure. What would you investigate?

Interview answer: I would suspect the threads are blocked or stalled rather than computing. Check major faults, swap-in/out, direct reclaim, PSI memory pressure, storage I/O latency, dirty-page writeback/throttling, `cgroup` memory events, OOM activity and NUMA placement.

Mechanism: Use `vmstat`, `pidstat`, `iostat`, `/proc/pressure/memory`, `/proc/vmstat`, `cgroup memory.stat/memory.events`, `perf`, and request traces. Also inspect THP compaction and whether synchronous reclaim/compaction is occurring in allocation paths.

Principal-level point: Low CPU plus high latency is a strong signal to look for waiting: memory reclaim, I/O, locks, scheduler delays or remote memory rather than optimizing compute.

45. A 100-GB database gets slower after switching from huge pages to 4-KB pages. Why might that happen?

Interview answer: The database now needs far more translations to cover the same 100-GB working set. TLB reach collapses, TLB misses and page-table walks increase, page tables consume more memory, and translation overhead can become a significant fraction of memory-access cost.

Mechanism: With 2-MB pages, 100 GB needs roughly 51,200 page mappings; with 4-KB pages it needs about 26.2 million. The actual TLB has far fewer entries than either count, but huge pages let each cached translation cover 512 times more memory.

Principal-level point: Validate the hypothesis with hardware counters for dTLB/iTLB misses and page walks, plus database latency and CPU-stall metrics. Also check NUMA placement and whether the huge-page change altered memory placement or reclaim behavior.

High-Value Comparison Tables

Event	Kernel trap?	Storage I/O?	Typical cost
TLB miss	Usually no	No	Low to moderate
Minor page fault	Yes	No required page read	Moderate
Major page fault	Yes	Yes, required page must be read	Potentially very high

Metric	Meaning	Interview caution
VSZ	Total mapped virtual address space	Can be huge without equivalent RAM usage
RSS	Resident mapped pages charged to process	Shared pages and allocator retention complicate interpretation
PSS	Shared pages divided proportionally among processes	Often better for shared-memory attribution
Private dirty / anonymous	Private process memory likely to require backing	Useful when investigating heap growth

Principal Engineer Production Checklist

When latency rises under memory pressure, correlate: application latency -> PSI memory pressure -> minor/major faults -> swap/reclaim -> block I/O -> cgroup events -> NUMA locality -> TLB/page-walk counters. Avoid concluding 'memory leak' or 'not enough RAM' from RSS/free memory alone.

Useful Linux commands:

```
vmstat 1
pidstat -r -p <pid> 1
cat /proc/<pid>/smaps_rollup
cat /proc/pressure/memory
cat /proc/vmstat
numastat -p <pid>
perf stat -p <pid> -e page-faults,minor-faults,major-faults
iostat -xz 1
```

Exact perf hardware event names for TLB misses/page walks vary by CPU model. Use `perf list` on the target host rather than memorizing one vendor-specific event name.

60-Second Revision

Remember: Virtual memory is an abstraction enforced by the MMU and controlled by OS page tables. TLBs cache translations. A TLB miss triggers translation work; a page fault traps to the OS. Demand paging and COW deliberately use faults. mmap connects virtual ranges to anonymous or file-backed objects. Page cache holds file data. Under pressure, reclaim and I/O can dominate latency. On multicore systems, TLB shootdowns and NUMA placement become scalability concerns. Huge pages trade granularity for greater TLB reach. In production, classify memory before diagnosing it.